

Plano de migração: BelaGo para monorepo TypeScript (web + api)

Ponto de partida

- `index.html` tem 3830 linhas, com cerca de 108 funções globais e 36 telas, sub-painéis e overlays.
- A integração com o Supabase é mínima e opcional, cerca de 6 chamadas: `professional_profiles`, `professional_services`, `profiles`, `appointments` e `account_deletion_requests`. Sem configuração, o app roda em modo demo com dados mockados.
- O schema já tem 16 tabelas com RLS: profiles, appointments, payments, payouts, disputes, reviews e outras.
- O deploy hoje é estático no Vercel, com um repo separado, `belago-producao`.

O front quase não depende de back-end. Por isso a migração é sobretudo **modularizar o front**, e a API entra depois, só onde o Supabase sozinho não resolve.

Estrutura alvo

```
belago/
|-- apps/
|   |-- web/           React + Vite + TypeScript
|   `-- api/          Fastify (ou Hono) + TypeScript
|-- packages/
|   `-- shared/        tipos, schemas zod, constantes (roles, status, especialidades)
|-- supabase/          schema.sql -> migrations/
|-- docs/
|-- package.json       workspaces (pnpm recomendado)
`-- CLAUDE.md          reescrito para a nova stack
```

Fase 0: Preparação (0,5 dia)

1. Criar a branch `migracao-monorepo`. Manter o `index.html` intacto até o fim da Fase 4, como referência de comportamento e fallback.
2. Decidir o gerenciador de pacotes (pnpm) e a versão do Node, e fixar em `.nvmrc`.
3. Reescrever o `CLAUDE.md`. Hoje ele proíbe frameworks, bundler e arquivos separados, e isso precisa mudar antes de qualquer código.
4. Confirmar o que fazer com o Capacitor citado no README, já que ele empacota o front.

Entrega: branch e CLAUDE.md novo aprovados.

Fase 1: Esqueleto do monorepo (1 dia)

1. Configurar o workspace raiz com scripts `dev`, `build`, `lint` e `typecheck`.

2. Criar `packages/shared` com os tipos base derivados do schema: `Role`, `Profile`, `ProfessionalProfile`, `Service`, `Appointment`, `AvailabilitySlot`.
3. Criar `apps/web` (Vite + React + TS) com uma tela em branco.
4. Criar `apps/api` com um `GET /health`.
5. Configurar ESLint, Prettier e `tsconfig.base.json`.
6. Gerar os tipos do banco com `supabase gen types` para alimentar o `shared`.

Entrega: `pnpm dev` sobe web e api juntos, e o web importa um tipo do `shared`.

Fase 2: Fundação do front (2 a 3 dias)

Esta fase substitui o núcleo do `index.html`.

1. **Roteamento:** trocar `goTab()` e `.screen / .active` por React Router, com rotas protegidas por papel (`cliente`, `profissional`, `admin`).
2. **Layouts:** uma tab bar por perfil (`ClienteLayout`, `ProfLayout`, `AdminLayout`). Isso elimina o risco de misturar as três tab bars.
3. **Design system:** migrar as variáveis CSS (paleta, `--r-*`, sombras) para `tokens.css` e criar os componentes base: `Button`, `Card`, `MenuRow`, `Toast`, `Overlay` (no lugar de `open0v/close0v`), `TabBar`.
4. **Auth:** criar `AuthContext` no lugar de `currentUser` e `routeUser()`, com `doLogout` como ação do contexto.
5. **Camada de dados:** criar `services/` com uma interface `DataSource`. Ela tem duas implementações, `mock` (o modo demo atual) e `supabase`, alternadas por variável de ambiente.
6. **Config:** trocar `window.BELAGO_CONFIG` por `.env` (`VITE_SUPABASE_URL` e `VITE_SUPABASE_ANON_KEY`).

Entrega: login com as 3 contas demo, redirecionamento por papel e logout funcionando.

Fase 3: Migração das telas (5 a 8 dias)

Migrar em ordem de valor e de dependência. Cada item tem um PR próprio e é comparado lado a lado com o `index.html`.

Ordem	Bloco	Conteúdo
1	Cliente: descoberta	<code>s-home</code> , <code>s-search</code> , <code>s-profile</code> (perfil do profissional)
2	Cliente: agendamento	fluxo de agendar, listagem de agendamentos, avaliação
3	Cliente: perfil	padrão hero + 3 stats + menu-rows
4	Profissional: Agenda	<code>ag-proximos</code> , <code>ag-realizados</code> , <code>ag-disponibilidade</code> (grade 06:00–22:30, <code>_availData</code>)
5	Profissional: resto	<code>pt-financeiro</code> , <code>pt-servicos</code> , <code>pt-perfil</code>
6	Admin	<code>at-painel</code> , <code>at-profissionais</code> , <code>at-clientes</code> , <code>at-financeiro</code> , <code>at-config</code> , <code>at-perfil</code>
7	Páginas estáticas	<code>legal.html</code> vira rota <code>/legal</code>

Regras durante a migração:

- Manter as regras de negócio do CLAUDE.md, por exemplo: sem headers nos painéis de profissional e admin, e sem emojis.
- Os dados `_adminProfs`, `_adminClients` e `_availData` viram fixtures do `DataSource` mock.

Entrega: paridade visual e funcional com o protótipo, em modo demo.

Fase 4: Integração real com o Supabase (2 a 3 dias)

1. Implementar o `DataSource` do Supabase para as tabelas já usadas: profissionais, serviços, perfil, agendamentos e solicitação de exclusão de conta.
2. Converter o `schema.sql` em `supabase/migrations/` versionadas.
3. Escrever `seed.sql` com as 3 contas demo, para o ambiente de dev ter o mesmo comportamento do mock.
4. Revisar as políticas RLS contra os fluxos reais. Por exemplo, `appt_participants_update` permite que qualquer participante altere o agendamento, e vale checar quais campos e transições de status cada papel pode mudar.

Entrega: app funcionando ponta a ponta com dados reais, sem o modo demo.

Fase 5: A API (3 a 5 dias)

O Supabase com RLS cobre o CRUD. A API só entra onde há lógica que não pode rodar no cliente:

Endpoint	Motivo
POST /appointments	validar disponibilidade, evitar conflito de horário e calcular comissão no servidor
Webhooks de pagamento	exigem segredo de servidor
POST /payouts (admin)	repasse e conciliação
DELETE /account	processar account_deletion_requests (LGPD)
Notificações	e-mail e push disparados por evento

1. Validar o JWT do Supabase no middleware da API.
2. Validar as entradas com os schemas zod do `packages/shared`, os mesmos usados no front.
3. Usar a `service_role key` só na API, nunca no front.
4. Escrever testes de integração com Vitest para os endpoints críticos.

Entrega: o front chama a API nas operações sensíveis e o Supabase direto no resto.

Fase 6: Deploy e virada (1 a 2 dias)

1. Configurar dois projetos Vercel a partir do mesmo repo (`apps/web` e `apps/api`), cada um com seu Root Directory. Alternativa: hospedar a API no Railway ou Render se ela precisar de processo persistente.
2. Migrar o `vercel.json`, mantendo os headers de segurança (`X-Frame-Options`, `nosniff`) e o fallback de SPA.
3. Decidir o destino do `belago-producao`. A sugestão é usar branches ou ambientes do Vercel (preview e production) e aposentar o repo separado, ou mantê-lo como espelho de release.
4. Configurar CI no GitHub Actions com `typecheck`, `lint`, `test` e `build`.
5. Remover o `index.html` legado.
6. Se o Capacitor continuar, gerar o build nativo a partir de `apps/web/dist`.

Entrega: produção rodando no novo monorepo.

Cronograma estimado

Fase	Duração
0 Preparação	0,5 dia
1 Esqueleto	1 dia
2 Fundação do front	2–3 dias
3 Telas	5–8 dias
4 Supabase real	2–3 dias
5 API	3–5 dias
6 Deploy	1–2 dias
Total	~15–23 dias úteis

Riscos e mitigação

- **Regressão visual:** comparar cada tela lado a lado com o `index.html` antes de fechar o PR.
- **Escopo crescendo:** congelar novas features no protótipo durante a migração, ou aplicá-las nos dois lados só se forem críticas.
- **RLS mal configurada:** testar cada política com um usuário de cada papel antes do deploy. A anon key fica exposta no front por natureza, então a RLS é a única barreira.
- **Duplicar lógica no front e na API:** tudo que for regra de negócio (comissão, status válidos) vive em `packages/shared` e é reusado.

Decisões pendentes

1. **React, ou outro framework de front?** Recomendação: React, pelo ecossistema e pela facilidade de contratar.
2. **Hospedagem da API:** Vercel (serverless, mais simples) ou Railway/Render (servidor persistente, melhor para filas e webhooks)?
3. **belago-producao** : aposentar ou manter?
4. **Capacitor:** o app nativo continua no escopo?